Regularized, Calibrated, and Robust Linear Models in StatCore: Algorithms and Reliability Evaluation

Jabulani Monchwe
Applied Business Analytics and Decision Making
North-West University
IDEA-Research Club Chairperson
Johannesburg, South Africa
monchwej.25@gmail.com

Martin Chanza
Applied Business Analytics and Decision Making
North-West University
Mafikeng, South Africa
martin.chanza@nwu.ac.za

Olorato Motitswane
Eduvos Midrand Campus – IT Faculty Data Science, South Africa
IDEA-Research Club
oloratomotitswane@gmail.com

Mfundo Monchwe
Eduvos Midrand Campus – IT Faculty Data Science, South Africa
IDEA-Research Club
diditmfundo@gmail.com

***Abstract*—We give a code-faithful account of the NumPy-based StatCore multinomial logistic classifier and scalar ridge regressor (the classifier uses SciPy L-BFGS-B; ridge uses NumPy linear algebra), each exposing `base`, `risk_calibrated`, `stat_regularised`, and `robust_loss`. For logistic regression, we derive the weighted softmax cross-entropy objective, its gradient, temperature fitting, cross-validated penalty selection, and the finite reweighting procedure for high-loss cases. For ridge regression, we derive the regularized normal equations, split-based linear recalibration and residual intervals, the implemented generalized cross-validation (GCV) criterion, and Huber-type iteratively reweighted least squares. We distinguish theoretical properties of the underlying estimators from properties the code does not ensure, including calibration, valid conformal coverage, global convergence, or robustness. Five-seed comparisons against scikit-learn standard references show that gains depend strongly on the scenario: logistic robust loss improves the deliberately noisy/outlier classification cases but can harm clean log-loss, while robust ridge is notably helpful under response outliers. The paper reports negative and neutral results as well as gains.**

***Index Terms*—logistic regression, ridge regression, probability calibration, robust estimation, generalized cross-validation, conformal prediction**

I. Introduction

Multinomial logistic regression estimates conditional class probabilities through the softmax of affine scores, while ridge regression solves penalized least squares with an unpenalized intercept [1], [2]. Both are useful linear references, but their reliability depends on regularization, outliers, probability calibration, and the split design. StatCore provides NumPy-based estimators with a SciPy L-BFGS-B optimizer for logistic regression and direct linear solves for ridge. This paper formalizes the exact behavior in `src/ml_reliability/statcore/linear/linear.py`; the four variant names are implementation options, not claims of new estimators.

II. Data contract and notation

The shared validator converts inputs to finite floating-point matrices, requires two-dimensional X, at least two training rows, a nonempty feature dimension, and matching one-dimensional labels. Logistic labels are mapped in sorted `numpy.unique` order; at least two classes are required. Ridge converts responses to finite floats. For all modes, `Standardizer` uses column means and population standard deviations computed over the full X passed to `fit`; any scale $\le 10^{-12}$ is replaced by 1. Thus $z_{ij} = (x_{ij} - \bar{x}_j)/s_j$. For split-based variants, scaling occurs before splitting, so validation covariates contribute to the fitted scaling statistics. This is an implementation detail and a source of mild information leakage in the internal split.

III. Multinomial logistic regression

A. Reference method and objective

Let C be the class count, $Z \in \mathbb{R}^{n\times d}$ standardized features, $Y \in \{0,1\}^{n\times C}$ one-hot responses, $W \in \mathbb{R}^{C\times d}$, and $b \in \mathbb{R}^C$. The logits and class probabilities are

$$\eta_i = W z_i + b, \qquad p_{ic} = \frac{\exp(\eta_{ic})}{\sum_{r=1}^{C} \exp(\eta_{ir})}. \tag{1}$$

For nonnegative sample weights a_i, define $\bar{a}_i = a_i / \sum_j a_j$; default weights are one. StatCore minimizes

$$J(W,b) = -\sum_{i=1}^{n} \bar{a}_i \sum_{c=1}^{C} Y_{ic} \log p_{ic} + \frac{\alpha}{2}\|W\|_F^2. \tag{2}$$

The intercept is not penalized. The default $\alpha = 10^{-2}$, `max_iter`=300. For an unweighted fit, loss is mean NLL, not total NLL. Stable softmax subtracts each row maximum, and log probabilities in the objective are clipped below at 10^{-300}. Given $G_{ic} = \bar{a}_i(p_{ic} - Y_{ic})$, differentiation yields

$$\nabla_W J = G^\top Z + \alpha W, \qquad \nabla_b J = \sum_i G_i. \tag{3}$$

The implementation initializes all parameters at zero and calls SciPy `minimize(method="L-BFGS-B", jac=True)` with `maxiter`; it does not inspect or expose the optimizer's convergence flag. Convexity of (2) for $\alpha \geq 0$ gives a global minimizer (though it need not be unique in unregularized degenerate data), but finite L-BFGS-B termination does not prove that it reached one. No explicit validation rejects negative `alpha`; for a negative penalty the objective need not be convex or bounded.

1) Fit the population-mean/standard-deviation scaler and encode sorted class labels as one-hot rows.
2) Initialize $(W, b) = 0$; evaluate (2) and gradient (3) in each L-BFGS-B step, up to `max_iter`.
3) Store class coefficients, intercept, and the selected variant's temperature (one by default).
4) At prediction, standardize with the stored scaler, compute affine logits divided by temperature, and return softmax, maximum probability, or the sorted class at the largest logit.

B. `risk_calibrated`: held-out scalar temperature

This mode partitions indices using the shared seeded splitter, stratified by encoded label, with default `calib_fraction`=0.25 and `random_state`=0. The classifier is fitted on the training indices only, using fixed α, then held-out logits $q_i = Wz_i + b$ are evaluated. For $u = \log T$, the scalar objective is

$$L(u) = -\frac{1}{|H|} \sum_{i \in H} \log \operatorname{softmax}(q_i / e^u)_{y_i}. \tag{4}$$

A 60-iteration golden-section search minimizes this function on $[-3, 3]$; therefore $T \in [e^{-3}, e^3]$. Temperature rescales logits but leaves their argmax unchanged for positive T, so it can change probability sharpness and log-loss but not accuracy (except ties/numerics). The classifier is not refit on calibration rows. This is a scalar calibration fit, not a guaranteed calibration procedure: one holdout, finite optimization range, small class counts, or distribution changes can yield worse calibration or log-loss. The random splitter keeps at least one row of a class in training when possible; it does not validate that `calib_fraction` is in $(0, 1)$.

1) Scale using all input rows, make one stratified train/holdout split, and fit (2) on the split's training subset.
2) Minimize (4) by 60 golden-section steps over $u \in [-3, 3]$; store $T = e^{\hat{u}}$.
3) Divide inference logits by T, then apply softmax. Do not refit coefficients on the holdout.

The fit uses one optimizer run plus 60 operations on cached holdout logits, the latter costing $O(60|H|C)$. The train fit costs approximately $O(IndC)$ for $I \leq 300$ iterations, plus L-BFGS-B memory linear in parameter count times its internal history.

C. `stat_regularised`: stratified cross-validation over α

The default grid is $\{10^{-4}, 10^{-3}, 10^{-2}, 10^{-1}, 1, 10\}$; a user-provided `alpha_grid` must be nonnegative. Fold count is implemented as

$$K = \max\left(2, \min\left(\mathrm{cv},\ \max(2, n_{\min}),\ n\right)\right), \tag{5}$$

where $n_{\min}$ is the smallest class count. The shared deterministic stratifier distributes shuffled class indices cyclically across fold IDs. For each grid value, one fit per fold uses `max_iter`=100; selection minimizes total held-out categorical NLL divided by n. The selected value is stored as `alpha_`, then the final estimator is refitted on all rows using `max_iter`. The scaler is fitted once before CV, using all rows, rather than refit within each fold. Thus score selection includes a small unsupervised validation-feature leak and its scores are not an outer estimate of generalization. With fewer than two rows in a class, the nominal minimum $K = 2$ may leave validation folds lacking that class; classes are nevertheless encoded from full training labels. The grid and CV controls are `alpha_grid=None`, `cv=5`, and `random_state=0`.

This is discrete empirical log-loss selection, not a closed-form statistical regularizer. For G grid values, cost is approximately GK fits of up to 100 iterations plus the final fit; default $G = 6, K \leq 5$. It provides no unbiased performance estimate after selecting the minimum. If a grid is empty or CV parameters are invalid, failures can occur in array selection/fold generation.

1) Fit one scaler on all supplied training features, encode labels, determine K by (5) and build fixed seeded stratified folds.
2) For each nonnegative α, fit each fold with 100 L-BFGS-B iterations and sum held-out NLL; select the lowest average-per-row score.
3) Refit with selected α and `max_iter`; expose the score mapping as `cv_scores_`.

D. `robust_loss`: finite high-NLL reweighting

This variant starts with $a_i^{(0)} = 1$ and performs exactly six outer rounds, each fitting (2) with normalized current weights. For fitted true-label probabilities, define $\ell_i = -\log \max(p_{i,y_i}, 10^{-12})$. Let $m_\ell = \operatorname{median}(\ell_i)$, $s_\ell = 1.4826 \operatorname{median} |\ell_i - m_\ell| + 10^{-6}$, and cutoff $c = m_\ell + 1.345 s_\ell$. The next weights are

$$a_i^{(t+1)} = \begin{cases} 1, & \ell_i \leq c, \\ c/\ell_i, & \ell_i > c. \end{cases} \tag{6}$$

The code sets the final `sample_weights_` to the last updated vector, but its final stored coefficients were fitted using the weights from the preceding round. Reweighting discounts observations the current classifier predicts with large NLL; it is Huber-type rather than an exact minimizer of a fixed Huber

loss, because the scale/cutoff and weights are recomputed from current residuals and the number of rounds is fixed. There is no convergence test or breakdown-point guarantee. Numerically, NumPy's vectorized `where` computes c/ℓ_i even on the inlier branch; a zero NLL can emit a divide-by-zero warning although the selected weight is one. The mode costs at most six optimizer runs, or about 6×300 L-BFGS-B iterations.

1) Initialize unit sample weights.
2) Repeat six times: fit weighted cross-entropy (2), compute in-sample true-class NLLs, robust median/MAD scale, and weights (6).
3) Retain the coefficients from the sixth fit and the weights produced after it; use the ordinary softmax at inference.

IV. Ridge regression

A. Reference estimator and closed form

Let z_i be standardized feature rows, $A = [\mathbf{1}, Z]$, $\beta = (b, w^\top)^\top$. For weights a_i, StatCore's solver returns

$$\begin{aligned}\hat\beta &= (A^\top D_a A + \alpha P + 10^{-10} I_{d+1})^{-1} A^\top D_a y,\\ P &= \operatorname{diag}(0, 1, \ldots, 1),\end{aligned} \tag{7}$$

implemented by `numpy.linalg.solve` (not an explicit matrix inverse). This is the unique minimizer, when the system is positive definite, of

$$\sum_i a_i (y_i - b - w^\top z_i)^2 + \alpha\|w\|_2^2 + 10^{-10}(b^2 + \|w\|_2^2). \tag{8}$$

The small stabilizer technically penalizes the intercept too, despite the main ridge penalty leaving it unpenalized. With $\alpha = 1$ and unit weights this is the base variant. Differentiating the quadratic objective gives (7); the positive diagonal stabilizer makes the augmented normal matrix positive definite in exact arithmetic. The solve costs $O(nd^2 + d^3)$ with $O(d^2)$ storage; forming the Gram matrix is often the bottleneck for large d, and no iterative sparse solver is used.

1) Standardize from all input features; append an intercept column.
2) Form the weighted Gram matrix and right-hand side; add $\alpha P + 10^{-10} I$.
3) Solve the linear system; store feature coefficients and intercept. Inference is affine in standardized input.

B. `risk_calibrated`: split recalibration and residual interval

Using the unstratified seeded holdout splitter with default fraction 0.25, coefficients are fitted only on training rows. On holdout prediction q_i, when there are at least three holdout rows and $\operatorname{sd}(q) > 10^{-12}$, the code computes

$$b_c = \frac{\widehat{\operatorname{Cov}}_{\text{pop}}(q, y)}{\operatorname{Var}_{\text{pop}}(q)}, \qquad a_c = \bar y - b_c \bar q, \tag{9}$$

with population covariance/variance, and inference returns $a_c + b_c q$. Otherwise it keeps $(a_c, b_c) = (0, 1)$. It then sorts absolute residuals $|y_i - (a_c + b_c q_i)|$ from those same holdout rows. At requested level $\ell \in (0, 1)$, with m residuals, the reported radius is

$$k = \min(\lceil (m+1)\ell \rceil, m) - 1, \quad q_\ell = r_{(\max(k,0))}, \tag{10}$$

using zero-based indexing in code; returned interval is prediction $\pm q_\ell$. This exact quantile is clipped to the largest available residual when ℓ is high. It resembles split-conformal intervals [5], but it is not a valid split-conformal construction as implemented: the same holdout labels fit the affine calibration map and estimate residual quantiles. The residuals therefore are not exchangeable with a new residual conditional on that data-dependent map. No finite-sample coverage guarantee is claimed. A deployment shift also invalidates exchangeability. `predict_interval` is allowed only for this variant and checks $0 < \ell < 1$; it does not check fitted state before calling `predict`.

1) Compute full-data scaler; randomly split rows; solve ridge on the training indices.
2) Fit (9) on holdout predictions if the coded size/variance conditions hold.
3) Store sorted residuals after calibration on the same holdout. At inference, use (10) for a requested level.

C. `stat_regularised`: one-shot GCV selection

This mode computes a centered SVD of standardized full-data design and centered response: $Z_c = Z - \bar Z = U \operatorname{diag}(s) V^\top$, $y_c = y - \bar y$. For each λ in user `alpha_grid` or default `numpy.logspace(-6,3,46)`, set $h_j = s_j^2/(s_j^2 + \lambda)$. The implementation evaluates

$$\operatorname{RSS}(\lambda) = \|y_c\|^2 - \|U^\top y_c\|^2 + \sum_j [(1 - h_j) u_j^\top y_c]^2, \tag{11}$$

$$\operatorname{df}(\lambda) = 1 + \sum_j h_j, \tag{12}$$

$$\operatorname{GCV}(\lambda) = \frac{n \operatorname{RSS}(\lambda)}{\max(n - \operatorname{df}(\lambda), 10^{-8})^2}. \tag{13}$$

The first residual term is response energy orthogonal to the thin-SVD column space; the extra one in degrees of freedom is the unpenalized intercept. The selected minimizing grid value is used in (7) on all rows. This is the standard linear-smoother GCV idea [3], specialized to the code's centered design and penalty convention. There is no fold CV. The singular values are computed once in $O(\min(n,d)^2 \max(n,d))$; scoring all $G = 46$ values is $O(G \min(n,d))$. User-provided grid values are not checked for nonnegativity; the default is positive. For an empty grid, no best candidate is produced. GCV is an approximate prediction-risk criterion, not an unbiased external test estimate, and the shared scaler is computed before the centered SVD using all training rows.

1) Standardize features, center Z and y, and compute a thin SVD once.
2) For every grid λ, evaluate (13) and choose the lowest score.
3) Store `alpha_` and `gcv_`; solve (7) on all observations.

D. `robust_loss`: Huber-type IRLS

Starting from unit weights and fixed $\lambda = \texttt{alpha}$, up to 30 iterations solve (7), calculate residuals $r_i = y_i - b - w^\top z_i$, and estimate robust scale

$$\begin{aligned} \hat{\sigma} &= 1.4826 \ \mathrm{median}_i \, |r_i - \mathrm{median}(r)| + 10^{-12}, \\ c &= \texttt{huber_k}\, \hat{\sigma}. \end{aligned} \tag{14}$$

Weights update to

$$a_i^{new} = \begin{cases} 1, & |r_i| \le c, \\ c/\max(|r_i|, 10^{-12}), & |r_i| > c. \end{cases} \tag{15}$$

The default `huber_k`=1.345, `alpha`=1. Convergence is declared when $\max_i |a_i^{new} - a_i| < 10^{-6}$; otherwise the loop ends after 30 solves. Coefficients from the solve before the final weight update are stored, and `sample_weights_` holds the newly updated weights, so stored weights need not be the exact weights used in the final solve. For fixed scale, Huber IRLS weights correspond to a bounded-influence score [4], but the implementation re-estimates MAD each iteration and uses a finite stopping rule; it does not verify objective decrease or convergence to a global optimum. The code checks nonnegative `alpha` but does not validate `huber_k`; a negative value could produce invalid negative weights. Each step costs the dense weighted solve $O(nd^2 + d^3)$, at most 30 times.

1) Initialize $a_i = 1$; solve the weighted ridge system.
2) Compute centered-MAD scale and update weights by (15).
3) Stop when the max weight change is below 10^{-6}, or after 30 rounds; store the coefficients from the last solve.

V. Methods side-by-side

VI. Experimental protocol

The reproducible entry point is `docs/latex/experiments/linear_experiment.py`; run it from any working directory with the repository's `.venv` `Scripts` `python.exe`. It evaluates all four StatCore variants and the standard references using five fixed seeds (11, 23, 37, 51, 79). Scikit-learn is used only for standard reference estimators (scaled logistic regression and scaled Ridge) and its bundled Breast Cancer and diabetes data [6]. The logistic reference uses $C = 1/(n_{train} \times 0.01)$, matching the order of the StatCore mean-loss penalty under the usual sum-loss sklearn parameterization; this does not make optimizer conventions identical. Ridge reference uses $\alpha = 1$. Baseline scalers are fitted within their training pipelines.

Classification synthetic experiments use a 12-feature logistic data generator (360 train/600 test); high-dimensional uses 80 features, 10 informative (140/300). Conditions are clean, target-feature mean shift (test only), 15% training label flips, and 8% training feature outliers drawn from $N(0, 6^2 I)$. The bundled Breast Cancer Wisconsin Diagnostic dataset is randomly split 75/25 for each seed. Regression uses a 12-feature linear model (360/600) with standard-normal features and unit test noise; the high-dimensional version uses 80 features and 100/300 rows; shifted test features have nonzero means; noisy training response noise has standard deviation 3; response-outlier condition adds noise of SD 20 to 8% of training responses. The bundled diabetes dataset is randomly split 75/25. Synthetic generation and split logic are in the script.

Held-out classification metrics are accuracy, natural log-loss and 10-bin equal-width confidence ECE, $\sum_b (n_b/n)|\mathrm{acc}_b - \mathrm{conf}_b|$, based on maximum class probability; ECE is descriptive and can depend on binning. Ridge reports MSE, MAE and $R^2 = 1 - \mathrm{SSE}/\sum_i (y_i - \bar{y})^2$. Tables give means $\pm$ sample standard deviations across five independent seeds. These are single held-out comparisons per seed, not nested CV or formal hypothesis tests. The bundled dataset splits are not an external validation of deployment performance.

VII. Classification results

Table II presents four synthetic scenarios and Table III includes high-dimensional data and Breast Cancer. Entries are measured run values, rounded to four decimal places.

VIII. Regression results

Table IV summarizes controlled data and Table V covers the high-dimensional and bundled diabetes data. MSE and MAE are smaller-is-better, while R^2 is larger-is-better.

IX. Interpretation and limitations

For logistic regression, finite reweighting improves both accuracy and probability losses under synthetic label noise (accuracy .7450, NLL .5199, ECE .0450 versus base .7383/.5364/.0630) and feature outliers (.7453/.5124/.0383 versus .7163/.5724/.0722). In clean data, it has worse log-loss (.5654 versus .4864) and ECE (.1029 versus .0326), exposing its sensitivity to pointwise classification difficulty rather than a universal robustness advantage. The internal-temperature option lowers ECE in several cases but can seriously worsen log-loss on Breast Cancer (.2708 versus .1076); accuracy is largely unchanged by positive temperature. GCV-selected logistic regularization helps in the high-dimensional synthetic case but is weaker than base on the outlier case. A common standardizing step is calculated once, not nested within each CV fold.

For ridge, the robust IRLS variant reduces outlier-case MSE from 1.8954 to 1.0236 and raises R^2 from .7072 to .8410, whereas in clean/noisy response cases it is nearly tied or slightly worse. The ridge GCV variant is beneficial for the deliberately underdetermined high-dimensional experiment (MSE 2.5458 versus 4.2702; R^2 = .6469 versus .4148), but not on the small bundled Diabetes split (MSE 3136.7 versus 3128.8). Split recalibration does not improve regression error in these experiments; same-holdout residual reuse means its reported interval should not be treated as having guaranteed conformal coverage. These results are descriptive, not evidence of statistical significance.

TABLE I
STATCORE LINEAR-MODEL VARIANTS AND DEFAULTS EXACTLY AS IMPLEMENTED.

Model/mode	Fitting rule	Default controls	Main limitation
Logistic `base`	Mean multinomial NLL plus $\alpha\|W\|_F^2/2$, L-BFGS-B	$\alpha = .01$, max 300	No calibration or robust weighting
Logistic risk-calibrated	Fit on stratified subset, then golden-section temperature NLL	holdout .25, $u \in [-3, 3]$, 60 steps	Holdout not used to refit; calibration may worsen
Logistic stat-regularised	Stratified CV mean log-loss grid selection, final refit	grid $10^{-4} \dots 10$, $K = 5$; fold max 100	Scaler fitted before folds; selected CV score optimistic
Logistic robust-loss	Six rounds of NLL/MAD sample reweighting	1.4826 MAD, cutoff factor 1.345, 6 rounds	Heuristic finite loop, no robust-loss convergence proof
Ridge `base`	Normal equations with $\alpha P + 10^{-10} I$	$\alpha = 1$	Dense Gram solve; tiny intercept penalty
Ridge risk-calibrated	Holdout linear recalibration plus same-holdout residual radius	holdout .25; interval level input	Reused holdout invalidates standard split-conformal guarantee
Ridge stat-regularised	Thin-SVD GCV grid selection and full fit	46 values $10^{-6} \dots 10^{3}$	Approximate in-sample criterion; user grid unchecked
Ridge robust-loss	MAD-scaled Huber IRLS weighted normal equations	$\alpha = 1$, $k = 1.345$, 30 steps	Finite iterative reweighting, scale/data dependent

TABLE II
LOGISTIC CLASSIFICATION RESULTS: CLEAN, SHIFTED, LABEL-NOISY, AND OUTLIER SYNTHETIC CASES; MEAN ± SAMPLE SD (FIVE SEEDS).

Case	Estimator	Accuracy	Log-loss	ECE
clean	sklearn standard	.7623 ± .0147	.4867 ± .0166	.0306 ± .0136
	base	.7617 ± .0137	.4864 ± .0195	.0326 ± .0100
	risk-calibrated	.7610 ± .0104	.4900 ± .0168	.0316 ± .0047
	stat-regularised	.7617 ± .0137	.4864 ± .0195	.0326 ± .0100
	robust-loss	.7600 ± .0169	.5654 ± .0615	.1029 ± .0245
shifted	sklearn standard	.7593 ± .0196	.4919 ± .0214	.0362 ± .0108
	base	.7600 ± .0199	.4921 ± .0251	.0397 ± .0163
	risk-calibrated	.7557 ± .0147	.4940 ± .0233	.0368 ± .0068
	stat-regularised	.7600 ± .0199	.4921 ± .0251	.0397 ± .0163
	robust-loss	.7573 ± .0123	.5758 ± .0686	.1015 ± .0254
label-noise	sklearn standard	.7363 ± .0071	.5386 ± .0060	.0656 ± .0156
	base	.7383 ± .0066	.5364 ± .0064	.0630 ± .0152
	risk-calibrated	.7233 ± .0134	.5817 ± .0276	.1094 ± .0281
	stat-regularised	.7330 ± .0095	.5548 ± .0043	.0890 ± .0139
	robust-loss	.7450 ± .0044	.5199 ± .0186	.0450 ± .0177
outliers	sklearn standard	.7150 ± .0287	.5771 ± .0215	.0801 ± .0371
	base	.7163 ± .0277	.5724 ± .0218	.0722 ± .0352
	risk-calibrated	.7057 ± .0330	.6237 ± .0483	.1205 ± .0754
	stat-regularised	.6703 ± .0709	.6261 ± .0441	.0971 ± .0551
	robust-loss	.7453 ± .0172	.5124 ± .0186	.0383 ± .0104

The standard sklearn reference is not identical in every solver detail. Scaling is comparable; the logistic regularization parameter is mapped to match the mean-loss scale approximately, while solver defaults and stopping criteria differ. No hyperparameter or ECE-bin selection was based on held-out test labels. Accuracy and ECE can both obscure class-specific errors; future evaluation should report subgroup and operating-point metrics, paired tests across independent datasets, and outer validation when selecting variants.

X. CONCLUSION

The source has four variants per class, but not all names imply theoretical guarantees. The logistic risk option is a scalar post-hoc temperature fit; its statistical regularizer is fold-wise log-loss selection; its robust option is six adaptive reweighting rounds. Ridge's "risk" mode combines an affine calibration fit and an interval based on reused residuals, its "statistical" mode is a precise grid GCV calculation, and its robust mode is finite MAD-scaled IRLS. The equations and caveats here make those semantics explicit. The observed wins are tied to their corresponding assumptions and data designs; standard linear estimators remain competitive, and no reliability property follows from a variant label alone.

REFERENCES

[1] P. McCullagh and J. A. Nelder, *Generalized Linear Models*, 2nd ed. London, U.K.: Chapman & Hall, 1989.

[2] A. E. Hoerl and R. W. Kennard, "Ridge regression: Biased estimation for nonorthogonal problems," *Technometrics*, vol. 12, no. 1, pp. 55–67, 1970.

[3] G. H. Golub, M. Heath, and G. Wahba, "Generalized cross-validation as a method for choosing a good ridge parameter," *Technometrics*, vol. 21, no. 2, pp. 215–223, 1979.

[4] P. J. Huber, *Robust Statistics*. New York, NY, USA: Wiley, 1981.

[5] V. Vovk, A. Gammerman, and G. Shafer, *Algorithmic Learning in a Random World*. New York, NY, USA: Springer, 2005.

[6] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

TABLE III

LOGISTIC CLASSIFICATION RESULTS: HIGH-DIMENSIONAL AND BUNDLED BREAST CANCER CASES; MEAN ± SAMPLE SD (FIVE SEEDS).

Case	Estimator	Accuracy	Log-loss	ECE
high-dimensional	sklearn standard	.6340 ± .0286	1.0248 ± .0641	.2353 ± .0179
	base	.6327 ± .0362	1.2924 ± .1107	.2648 ± .0292
	risk-calibrated	.6293 ± .0155	.6833 ± .0601	.0779 ± .0580
	stat-regularised	.6627 ± .0114	.6520 ± .0368	.0889 ± .0377
	robust-loss	.6433 ± .0127	1.0957 ± .1103	.2335 ± .0203
Breast Cancer	sklearn standard	.9704 ± .0116	.1111 ± .0186	.0371 ± .0085
	base	.9704 ± .0092	.1076 ± .0189	.0333 ± .0146
	risk-calibrated	.9606 ± .0170	.2708 ± .2535	.0271 ± .0124
	stat-regularised	.9704 ± .0092	.1076 ± .0189	.0333 ± .0146
	robust-loss	.9352 ± .0168	.1858 ± .0514	.0392 ± .0184

TABLE IV

RIDGE REGRESSION RESULTS: CLEAN, SHIFTED, NOISY-RESPONSE, AND OUTLIER CASES; MEAN ± SAMPLE SD (FIVE SEEDS).

Case	Estimator	MSE	MAE	R^2
clean	sklearn standard	1.0133 ± .0647	.8074 ± .0274	.8426 ± .0066
	base	1.0133 ± .0647	.8074 ± .0274	.8426 ± .0066
	risk-calibrated	1.0369 ± .0543	.8144 ± .0220	.8389 ± .0045
	stat-regularised	1.0134 ± .0653	.8074 ± .0275	.8426 ± .0067
	robust-loss	1.0130 ± .0677	.8076 ± .0281	.8426 ± .0073
shifted	sklearn standard	1.0319 ± .0690	.8155 ± .0285	.8397 ± .0063
	base	1.0319 ± .0690	.8155 ± .0285	.8397 ± .0063
	risk-calibrated	1.0588 ± .0505	.8250 ± .0191	.8354 ± .0066
	stat-regularised	1.0314 ± .0703	.8153 ± .0290	.8398 ± .0065
	robust-loss	1.0303 ± .0721	.8153 ± .0301	.8400 ± .0077
noisy	sklearn standard	1.2485 ± .1369	.8964 ± .0469	.8066 ± .0118
	base	1.2485 ± .1369	.8964 ± .0469	.8066 ± .0118
	risk-calibrated	1.6901 ± .1889	1.0449 ± .0662	.7354 ± .0452
	stat-regularised	1.2331 ± .1515	.8920 ± .0544	.8090 ± .0154
	robust-loss	1.2634 ± .1620	.9010 ± .0564	.8045 ± .0135
outliers	sklearn standard	1.8954 ± .3196	1.0942 ± .0768	.7072 ± .0309
	base	1.8954 ± .3196	1.0942 ± .0768	.7072 ± .0309
	risk-calibrated	2.1169 ± .2973	1.1656 ± .0819	.6716 ± .0396
	stat-regularised	1.8301 ± .1883	1.0804 ± .0582	.7155 ± .0296
	robust-loss	1.0236 ± .0771	.8110 ± .0311	.8410 ± .0089

TABLE V

RIDGE REGRESSION RESULTS: HIGH-DIMENSIONAL AND BUNDLED DIABETES CASES; MEAN ± SAMPLE SD (FIVE SEEDS).

Case	Estimator	MSE	MAE	R^2
high-dimensional	sklearn standard	4.2702 ± 1.6136	1.6378 ± .3083	.4148 ± .1822
	base	4.2702 ± 1.6136	1.6378 ± .3083	.4148 ± .1822
	risk-calibrated	5.1271 ± 2.1774	1.7654 ± .3284	.3007 ± .2436
	stat-regularised	2.5458 ± .2799	1.2668 ± .0618	.6469 ± .0237
	robust-loss	3.7527 ± .8932	1.5349 ± .1935	.4821 ± .0953
Diabetes	sklearn standard	3128.8100 ± 410.4238	45.4471 ± 3.1316	.4868 ± .0630
	base	3128.8100 ± 410.4238	45.4471 ± 3.1316	.4868 ± .0630
	risk-calibrated	3242.3199 ± 439.0180	46.7776 ± 3.1828	.4683 ± .0671
	stat-regularised	3136.7135 ± 413.4326	45.4778 ± 3.1319	.4855 ± .0636
	robust-loss	3137.4466 ± 397.4468	45.4578 ± 2.8843	.4854 ± .0605